

48-Hour Interview Add-on: Complete Implementation Plan

Executive Summary

This plan extends the existing IKEA lakehouse demo with 7 targeted additions that prove core data engineering competencies: API ingestion, SQL modeling, scalable architecture, Delta governance, performance tuning, testing, and BI integration.

Timeline: 1-2 days

Branch: feature/interview-demo

Deliverables: 5 notebooks, 1 config file, 1 test suite, 3-4 screenshots, README updates

APIs We'll Use

Primary: DummyJSON (Retail Entities + Pagination)

- **Base URL:** `https://dummyjson.com`
- **Endpoints:**
 - Products: `/products?limit=100&skip=0`
 - Users: `/users?limit=100&skip=0`
 - Carts: `/carts?limit=100&skip=0`
- **Why:** Retail-shaped data, demonstrates pagination, schema validation, deduplication
- **Bronze tables:** `bronze.products_raw` , `bronze.users_raw` , `bronze.carts_raw`

Secondary: Frankfurter (FX Rates + Time-Series)

- **Base URL:** `https://api.frankfurter.dev`
- **Endpoint:** `/2024-01-01..2025-10-15?from=EUR&to=SEK,USD`
- **Why:** Enrichment data for currency conversion, time-window fetches, idempotent re-runs
- **Bronze table:** `bronze.fx_rates_raw`

Repository Structure (Additions)

```
IKEA-demo/
├─ notebooks/
│   └─ 07_ingest_api_data.py           # NEW: API ingestion (DummyJSON + Frankfurte
│   └─ 08_silver_api_transform.sql     # NEW: CTE-heavy SQL with keys
│   └─ 09_metadata_joins.py           # NEW: Config-driven joins
│   └─ 10_delta_time_travel.py        # NEW: Delete, history, rollback
│   └─ 11_perf_skew_broadcast.py       # NEW: Performance tuning demo
├─ config/
│   └─ joins.yml                       # NEW: Join metadata config
├─ tests/
│   └─ test_transforms.py             # NEW: pytest + chispa unit tests
├─ ops/
│   └─ postman_collection.json         # NEW: API validation exports
│   └─ powerbi_refresh.py             # NEW (optional): BI refresh stub
└─ docs/
    └─ screenshots/
        └─ api_validation.png         # NEW: Postman screenshot
        └─ delta_history.png          # NEW: DESCRIBE HISTORY output
        └─ spark_ui_skew.png          # NEW: Before fix
        └─ spark_ui_fixed.png         # NEW: After AQE/broadcast
    └─ IMPLEMENTATION_PLAN.md         # THIS FILE
```

STEP 1: API Ingestion to Bronze (APIs & Data Ingestion)

Objective

Demonstrate robust external data ingestion with validation, pagination, schema enforcement, and error handling.

Pre-Work: API Validation (No Code)

1. Open Postman or use curl
2. Test DummyJSON endpoints:

```
# GET with pagination
curl "https://dummyjson.com/products?limit=10&skip=0"
```

```
# POST with filter (illustrate POST vs GET)
curl -X POST "https://dummyjson.com/products/search" \
  -H "Content-Type: application/json" \
  -d '{"q": "phone", "limit": 10}'
```

3. Test Frankfurter:

```
curl "https://api.frankfurter.dev/2024-01-01..2024-01-10?from=EUR&to=SEK,USD"
```

4. Export Postman collection as ops/postman_collection.json

5. Screenshot one validated request → docs/screenshots/api_validation.png

Notebook: notebooks/07_ingest_api_data.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC ## 07_ingest_api_data
# MAGIC Ingest external API data (DummyJSON + Frankfurter) to bronze with:
# MAGIC - Pagination (DummyJSON)
# MAGIC - Retry logic
# MAGIC - Explicit schemas (no infer)
# MAGIC - Idempotent loads

# COMMAND -----

# Install dependencies (if not in cluster libraries)
%pip install requests retrying

# COMMAND -----

import requests
from retrying import retry
from pyspark.sql import functions as F
from pyspark.sql.types import *
from datetime import datetime, timedelta
import json

# COMMAND -----

# MAGIC %md
# MAGIC ### 1. DummyJSON Products with Pagination

# COMMAND -----

@retry(stop_max_attempt_number=3, wait_fixed=2000)
def fetch_dummyjson_page(endpoint, limit=100, skip=0):
    """Fetch a single page from DummyJSON with retry logic."""
    url = f"https://dummyjson.com/{endpoint}"
    params = {"limit": limit, "skip": skip}

    response = requests.get(url, params=params, timeout=10)
    response.raise_for_status()
    return response.json()

# Fetch all products with pagination
```

```

all_products = []
skip = 0
limit = 100

while True:
    print(f"Fetching products: skip={skip}, limit={limit}")
    data = fetch_dummyjson_page("products", limit=limit, skip=skip)

    products = data.get("products", [])
    if not products:
        break

    all_products.extend(products)

    # Check if we've fetched all
    total = data.get("total", 0)
    if skip + len(products) >= total:
        break

    skip += limit

print(f"✓ Fetched {len(all_products)} products")

# COMMAND -----

# Define explicit schema for products (schema-on-write)
products_schema = StructType([
    StructField("id", IntegerType(), False),
    StructField("title", StringType(), False),
    StructField("description", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("discountPercentage", DoubleType(), True),
    StructField("rating", DoubleType(), True),
    StructField("stock", IntegerType(), True),
    StructField("brand", StringType(), True),
    StructField("category", StringType(), True),
    StructField("thumbnail", StringType(), True),
    StructField("images", StringType(), True), # JSON array as string
])

# Convert to DataFrame with explicit schema
ingested_at = datetime.now().isoformat()
products_data = [

```

```
(
    p["id"],
    p["title"],
    p.get("description"),
    p.get("price"),
    p.get("discountPercentage"),
    p.get("rating"),
    p.get("stock"),
    p.get("brand"),
    p.get("category"),
    p.get("thumbnail"),
    json.dumps(p.get("images", [])),
)
for p in all_products
]
```

```
df_products = spark.createDataFrame(products_data, schema=products_schema)
df_products = df_products.withColumn("ingested_at", F.lit(ingested_at))
df_products = df_products.withColumn("source", F.lit("dummyjson.com/products"))
```

```
# Write to bronze with merge for idempotency
df_products.write.mode("overwrite").saveAsTable("bronze.products_raw")
```

```
print(f"✓ Loaded {df_products.count()} products to bronze.products_raw")
display(df_products.limit(5))
```

```
# COMMAND -----
```

```
# MAGIC %md
```

```
# MAGIC ### 2. DummyJSON Users (same pagination pattern)
```

```
# COMMAND -----
```

```
all_users = []
skip = 0
while True:
    print(f"Fetching users: skip={skip}")
    data = fetch_dummyjson_page("users", limit=100, skip=skip)
    users = data.get("users", [])
    if not users:
        break
    all_users.extend(users)
    if skip + len(users) >= data.get("total", 0):
```

```

        break
    skip += 100

# Simplified user schema
df_users = spark.createDataFrame([
    (
        u["id"],
        u.get("firstName"),
        u.get("lastName"),
        u.get("email"),
        u.get("age"),
        u.get("gender"),
        json.dumps(u.get("address", {})),
    )
    for u in all_users
], ["id", "first_name", "last_name", "email", "age", "gender", "address_json"])

df_users = df_users.withColumn("ingested_at", F.lit(ingested_at))
df_users.write.mode("overwrite").saveAsTable("bronze.users_raw")

print(f"✓ Loaded {df_users.count()} users to bronze.users_raw")

# COMMAND -----

# MAGIC %md
# MAGIC ### 3. Frankfurter FX Rates (time-series range)

# COMMAND -----

@retry(stop_max_attempt_number=3, wait_fixed=2000)
def fetch_fx_range(start_date, end_date, base="EUR", symbols="SEK,USD"):
    """Fetch FX rates for a date range."""
    url = f"https://api.frankfurter.dev/{start_date}..{end_date}"
    params = {"from": base, "to": symbols}

    response = requests.get(url, params=params, timeout=10)
    response.raise_for_status()
    return response.json()

# Fetch last 90 days
end_date = datetime.now().date()
start_date = end_date - timedelta(days=90)

```

```

print(f"Fetching FX rates: {start_date} to {end_date}")
fx_data = fetch_fx_range(start_date.isoformat(), end_date.isoformat())

# Convert nested JSON to rows
fx_rows = []
for date_str, rates in fx_data.get("rates", {}).items():
    for pair, rate in rates.items():
        fx_rows.append((
            date_str,
            fx_data["base"],
            pair,
            float(rate),
            ingested_at,
            f"https://api.frankfurter.dev"
        ))

fx_schema = StructType([
    StructField("as_of_date", StringType(), False),
    StructField("base_currency", StringType(), False),
    StructField("quote_currency", StringType(), False),
    StructField("rate", DoubleType(), False),
    StructField("ingested_at", StringType(), False),
    StructField("source_url", StringType(), True),
])

df_fx = spark.createDataFrame(fx_rows, schema=fx_schema)
df_fx = df_fx.withColumn("as_of_date", F.to_date("as_of_date"))

df_fx.write.mode("overwrite").saveAsTable("bronze.fx_rates_raw")

print(f"✓ Loaded {df_fx.count()} FX rate records to bronze.fx_rates_raw")
display(df_fx.limit(10))

# COMMAND -----

# MAGIC %md
# MAGIC ### Summary
# MAGIC - ✓ DummyJSON products: pagination, explicit schema, retry logic
# MAGIC - ✓ DummyJSON users: same pattern
# MAGIC - ✓ Frankfurter FX: time-range pull, flattened JSON
# MAGIC - All landed to bronze with `ingested_at` and `source` for lineage

```


What This Proves

- GET vs POST understanding (Postman validation)
- Pagination implementation (skip/limit pattern)
- Retry logic and timeout handling
- Schema-on-write (explicit types vs infer)
- Idempotent ingestion patterns
- Bronze layer conventions (metadata columns)

STEP 2: Transform with CTEs & Keys (SQL Modeling)

Objective

Demonstrate clean SQL reasoning, CTE structure, primary/composite keys, and referential integrity.

SQL File: notebooks/08_silver_api_transform.sql

```
-- Databricks notebook source
-- MAGIC %md
-- MAGIC ## 08_silver_api_transform
-- MAGIC Transform bronze API data to clean silver dimensions and facts with:
-- MAGIC - CTE-based logic
-- MAGIC - Explicit primary/composite keys
-- MAGIC - Referential integrity examples
-- MAGIC - Deduplication

-- COMMAND -----

-- MAGIC %md
-- MAGIC ### 1. Silver FX Daily (CTE pattern with dedup)

-- COMMAND -----

CREATE OR REPLACE TABLE silver.fx_rates_daily
COMMENT 'Daily FX rates with explicit composite PK: (pair, as_of_date)'
AS

-- CTE 1: Clean and standardize
WITH clean_rates AS (
    SELECT
        as_of_date,
        CONCAT(base_currency, '/', quote_currency) AS pair,
        rate,
        ingested_at,
        source_url
    FROM bronze.fx_rates_raw
    WHERE rate > 0 -- Data quality check
        AND as_of_date IS NOT NULL
        AND base_currency IS NOT NULL
        AND quote_currency IS NOT NULL
),

-- CTE 2: Deduplicate (take latest ingestion if duplicates)
deduped AS (
    SELECT
        as_of_date,
        pair,
        rate,
```

```

    ROW_NUMBER() OVER (
        PARTITION BY pair, as_of_date
        ORDER BY ingested_at DESC
    ) AS rn
FROM clean_rates
),

-- CTE 3: Keep latest only
latest_by_day AS (
    SELECT
        as_of_date,
        pair,
        rate
    FROM deduped
    WHERE rn = 1
)

SELECT
    pair,
    as_of_date,
    rate,
    CURRENT_TIMESTAMP() AS transformed_at
FROM latest_by_day
-- PRIMARY KEY (pair, as_of_date) -- Conceptual; enforced via Delta constraints if sup
;

-- COMMAND -----

-- Verify no duplicates on composite key
SELECT
    pair,
    as_of_date,
    COUNT(*) AS cnt
FROM silver.fx_rates_daily
GROUP BY pair, as_of_date
HAVING COUNT(*) > 1;

-- COMMAND -----

-- MAGIC %md
-- MAGIC ### 2. Silver Dim Currency (referential integrity example)

-- COMMAND -----

```

```

CREATE OR REPLACE TABLE silver.dim_currency
COMMENT 'Currency dimension with PK: currency_code'
AS
SELECT DISTINCT
    quote_currency AS currency_code,
    CASE
        WHEN quote_currency = 'SEK' THEN 'Swedish Krona'
        WHEN quote_currency = 'USD' THEN 'US Dollar'
        WHEN quote_currency = 'EUR' THEN 'Euro'
        ELSE quote_currency
    END AS currency_name
FROM bronze.fx_rates_raw
WHERE quote_currency IS NOT NULL
-- PRIMARY KEY (currency_code)
;

-- COMMAND -----

-- MAGIC %md
-- MAGIC ### 3. Silver Dim Products (from DummyJSON)

-- COMMAND -----

CREATE OR REPLACE TABLE silver.dim_products_api
COMMENT 'Product dimension from DummyJSON with PK: product_id'
AS

WITH clean_products AS (
    SELECT
        id AS product_id,
        title,
        description,
        category,
        brand,
        price,
        stock,
        rating,
        CASE
            WHEN price > 1000 THEN 'LARGE'
            ELSE 'SMALL'
        END AS size_class, -- Map to IKEA pattern
        ingested_at

```

```

    FROM bronze.products_raw
    WHERE id IS NOT NULL
),

deduped_products AS (
    SELECT
        *,
        ROW_NUMBER() OVER (PARTITION BY product_id ORDER BY ingested_at DESC) AS rn
    FROM clean_products
)

SELECT
    product_id,
    title,
    description,
    category,
    brand,
    price,
    stock,
    rating,
    size_class,
    CURRENT_TIMESTAMP() AS transformed_at
FROM deduped_products
WHERE rn = 1
-- PRIMARY KEY (product_id)
;

-- COMMAND -----

-- MAGIC %md
-- MAGIC ### 4. Silver Dim Customers (from DummyJSON users)

-- COMMAND -----

CREATE OR REPLACE TABLE silver.dim_customers_api
COMMENT 'Customer dimension from DummyJSON with PK: customer_id'
AS
SELECT
    id AS customer_id,
    CONCAT(first_name, ' ', last_name) AS customer_name,
    email,
    age,
    gender,

```

```

    CURRENT_TIMESTAMP() AS transformed_at
FROM bronze.users_raw
WHERE id IS NOT NULL
-- PRIMARY KEY (customer_id)
;

-- COMMAND -----

-- MAGIC %md
-- MAGIC ### Summary
-- MAGIC - ✓ CTEs used for readability (clean → dedup → latest)
-- MAGIC - ✓ Composite keys documented: (pair, as_of_date)
-- MAGIC - ✓ Primary keys documented: product_id, customer_id, currency_code
-- MAGIC - ✓ Referential integrity concept: fx_rates references dim_currency
-- MAGIC - ✓ Deduplication logic with window functions

```

What This Proves

- CTE structure for complex transforms
- Primary and composite key understanding
- Deduplication with ROW_NUMBER()
- Referential integrity (foreign key concept)
- Data quality checks (null filters, value ranges)

STEP 3: Metadata-Driven Joins (Architecture & Maintainability)

Objective

Show scalable, config-based join engine that handles 10-20 tables without new code.

Config File: config/joins.yml

```
# Metadata-driven join configuration
# Demonstrates: scalability, broadcast hints, maintainability

base:
  table: silver.baskets
  alias: base
  select:
    - receipt_id
    - loyalty_id
    - date
    - total_amount
    - item_count
  filter: "date >= '2025-01-01'"

joins:
  # Join 1: Customer dimension
  - table: silver.dim_customers_api
    alias: cust
    type: left
    on: "base.loyalty_id = cust.customer_id"
    select:
      - customer_id
      - customer_name
      - age
      - gender
    broadcast: true # Small dimension, use broadcast join

  # Join 2: FX rates for currency normalization
  - table: silver.fx_rates_daily
    alias: fx
    type: left
    on: "base.date = fx.as_of_date AND 'EUR/SEK' = fx.pair"
    select:
      - pair
      - rate
    broadcast: false # Time-series, might be large

  # Join 3: Product enrichment (example - would join via line items in real scenario)
  - table: silver.dim_products_api
    alias: prod
    type: left
```

```
on: "1=0" # Placeholder – real join would be via cart items
```

```
select:
```

- product_id
- category
- size_class

```
broadcast: true
```

```
output:
```

```
table: gold.baskets_enriched
```

```
mode: overwrite
```


Notebook: notebooks/09_metadata_joins.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC ## 09_metadata_joins
# MAGIC Metadata-driven join engine that:
# MAGIC - Reads join config from YAML
# MAGIC - Applies joins generically
# MAGIC - Honors broadcast hints
# MAGIC - Produces gold enriched tables

# COMMAND -----

%pip install pyyaml

# COMMAND -----

import yaml
from pathlib import Path
from pyspark.sql import functions as F

# COMMAND -----

# Load join configuration
def resolve_repo_root():
    try:
        return Path(__file__).resolve().parents[1]
    except NameError:
        try:
            nb_path = dbutils.notebook.entry_point.getDbutils().notebook().getContext()
            workspace_path = Path("/Workspace") / nb_path.lstrip("/")
            return workspace_path.parents[1]
        except:
            return Path.cwd().resolve()

repo_root = resolve_repo_root()
config_path = repo_root / "config" / "joins.yml"

if 'dbutils' in globals():
    # Read from DBFS
    config_content = dbutils.fs.head(f"dbfs:{config_path.as_posix()}", 10000)
else:
    # Read locally
```

```

    with open(config_path) as f:
        config_content = f.read()

config = yaml.safe_load(config_content)
print("✓ Loaded join configuration")
print(yaml.dump(config, default_flow_style=False))

# COMMAND -----

# MAGIC %md
# MAGIC ### Generic Join Engine

# COMMAND -----

def apply_metadata_joins(config):
    """
    Apply joins based on YAML config.

    Returns enriched DataFrame ready for gold layer.
    """

    # Step 1: Load base table
    base_cfg = config["base"]
    base_table = base_cfg["table"]
    base_alias = base_cfg.get("alias", "base")

    print(f"Loading base table: {base_table}")
    df_base = spark.table(base_table)

    # Apply base filter
    if "filter" in base_cfg:
        df_base = df_base.filter(base_cfg["filter"])
        print(f"    Applied filter: {base_cfg['filter']}")

    # Select base columns
    if "select" in base_cfg:
        df_base = df_base.select(*base_cfg["select"])
        print(f"    Selected {len(base_cfg['select'])} columns")

    df_base = df_base.alias(base_alias)
    result_df = df_base

    # Step 2: Apply joins sequentially

```

```

for idx, join_cfg in enumerate(config.get("joins", []), 1):
    join_table = join_cfg["table"]
    join_alias = join_cfg["alias"]
    join_type = join_cfg.get("type", "left")
    join_on = join_cfg["on"]
    broadcast = join_cfg.get("broadcast", False)

    print(f"\nJoin {idx}: {join_table} ({join_type})")

    # Load join table
    df_join = spark.table(join_table)

    # Select specific columns from join table
    if "select" in join_cfg:
        df_join = df_join.select(*join_cfg["select"])

    # Apply broadcast hint if specified
    if broadcast:
        df_join = F.broadcast(df_join)
        print(f"  ✓ Broadcast hint applied")

    df_join = df_join.alias(join_alias)

    # Perform join
    result_df = result_df.join(
        df_join,
        on=F.expr(join_on),
        how=join_type
    )

    print(f"  ✓ Joined on: {join_on}")

    return result_df

# COMMAND -----

# Execute join pipeline
df_enriched = apply_metadata_joins(config)

print(f"\n✓ Enriched DataFrame created")
print(f"  Rows: {df_enriched.count():,}")
print(f"  Columns: {len(df_enriched.columns)}")

```

```

display(df_enriched.limit(10))

# COMMAND -----

# Write to gold layer
output_cfg = config.get("output", {})
output_table = output_cfg.get("table", "gold.baskets_enriched")
output_mode = output_cfg.get("mode", "overwrite")

df_enriched.write.mode(output_mode).saveAsTable(output_table)

print(f"✓ Written to {output_table}")

# COMMAND -----

# MAGIC %md
# MAGIC ### Validation

# COMMAND -----

# Show join plan (verify broadcast worked)
df_enriched.explain("formatted")

# COMMAND -----

# MAGIC %md
# MAGIC ### Summary
# MAGIC - ✓ Config-driven joins (add 10-20 tables via YAML, not code)
# MAGIC - ✓ Broadcast hints honored (check explain plan)
# MAGIC - ✓ Generic join engine (DRY principle)
# MAGIC - ✓ Gold table created with enriched business context

```

What This Proves

- Architectural thinking (metadata over code)
- Maintainability (add tables via config)
- Broadcast join understanding
- DRY principle and code reusability

STEP 4: Delta Time Travel & Corrections (Governance)

Objective

Demonstrate Delta operations: delete, history, rollback, and auditability.

Notebook: notebooks/10_delta_time_travel.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC ## 10_delta_time_travel
# MAGIC Delta Lake governance features:
# MAGIC - DELETE operations
# MAGIC - DESCRIBE HISTORY
# MAGIC - Time travel (versionAsOf)
# MAGIC - Rollback patterns

# COMMAND -----

from delta.tables import DeltaTable
from pyspark.sql import functions as F

# COMMAND -----

# MAGIC %md
# MAGIC ### 1. Setup: Insert a "bad" row

# COMMAND -----

# Read current gold table
df_gold = spark.table("gold.baskets_enriched")

print(f"Current rows: {df_gold.count()}")

# Insert a known bad row (corrupted data scenario)
bad_row = spark.createDataFrame([
    (
        999999,      # receipt_id
        -1,          # loyalty_id (invalid)
        "2025-01-01",
        -9999.99,    # total_amount (corrupted)
        0,
        None,        # customer_name
        None,        # age
        None,        # gender
        None,        # pair
        None         # rate
    )
], schema=df_gold.schema)
```

```

bad_row.write.mode("append").saveAsTable("gold.baskets_enriched")

print(f"Rows after bad insert: {spark.table('gold.baskets_enriched').count()}")

# COMMAND -----

# Verify bad row exists
spark.sql("""
    SELECT * FROM gold.baskets_enriched
    WHERE receipt_id = 999999
""").show()

# COMMAND -----

# MAGIC %md
# MAGIC ### 2. DELETE Operation

# COMMAND -----

# Load as Delta table
delta_table = DeltaTable.forName(spark, "gold.baskets_enriched")

# Delete the bad row
delta_table.delete("receipt_id = 999999")

print("✓ Deleted bad row where receipt_id = 999999")

# Verify deletion
count_after_delete = spark.table("gold.baskets_enriched").count()
print(f"Rows after delete: {count_after_delete}")

# COMMAND -----

# MAGIC %md
# MAGIC ### 3. DESCRIBE HISTORY (Audit Trail)

# COMMAND -----

# Show full history of changes
history_df = delta_table.history()

print("=== DELTA TABLE HISTORY ===")

```

```

display(history_df.select(
    "version",
    "timestamp",
    "operation",
    "operationParameters",
    "operationMetrics"
))

# COMMAND -----

# MAGIC %md
# MAGIC 📸 **SCREENSHOT THIS OUTPUT** for README: `docs/screenshots/delta_history.png`

# COMMAND -----

# MAGIC %md
# MAGIC ### 4. Time Travel: Read Previous Versions

# COMMAND -----

# Get version numbers
versions = history_df.select("version").rdd.flatMap(lambda x: x).collect()
latest_version = max(versions)
previous_version = latest_version - 1

print(f"Latest version: {latest_version}")
print(f"Previous version: {previous_version}")

# COMMAND -----

# Read the version BEFORE the delete (when bad row still existed)
df_v_before_delete = spark.read.format("delta").option("versionAsOf", previous_version)

print(f"\nVersion {previous_version} (before delete) had {df_v_before_delete.count()} r

# Verify bad row exists in old version
df_v_before_delete.filter("receipt_id = 999999").show()

# COMMAND -----

# Read current version (after delete)
df_current = spark.table("gold.baskets_enriched")
print(f"Current version has {df_current.count()} rows")

```



```

# Verify bad row is gone
print(f"Bad row count in current: {df_current.filter('receipt_id = 999999').count()}")

# COMMAND -----

# MAGIC %md
# MAGIC ### 5. Rollback Pattern (Restore Previous Version)

# COMMAND -----

# If we wanted to rollback to previous version (hypothetically):
# Option A: RESTORE (Databricks SQL)
# spark.sql(f"RESTORE gold.baskets_enriched TO VERSION AS OF {previous_version}")

# Option B: Overwrite with old version
# df_v_before_delete.write.mode("overwrite").option("overwriteSchema", "false").saveAsT

print("""
Rollback options demonstrated:
1. RESTORE TABLE ... TO VERSION AS OF {version}
2. Read old version and overwrite current table
3. Read timestampAsOf for point-in-time recovery
""")

# COMMAND -----

# MAGIC %md
# MAGIC ### 6. Timestamp-Based Time Travel

# COMMAND -----

# Read as of a specific timestamp
import datetime
two_hours_ago = (datetime.datetime.now() - datetime.timedelta(hours=2)).strftime("%Y-%m

try:
    df_ts = spark.read.format("delta").option("timestampAsOf", two_hours_ago).table("go
    print(f"✓ Read table as of {two_hours_ago}")
    print(f"  Rows: {df_ts.count()}")
except Exception as e:
    print(f"⚠ No data at {two_hours_ago} (expected for new demo)")

```

```
# COMMAND -----
```

```
# MAGIC %md
```

```
# MAGIC ### Summary
```

```
# MAGIC - ✓ DELETE operation executed and verified
```

```
# MAGIC - ✓ DESCRIBE HISTORY shows full audit trail (screenshot this!)
```

```
# MAGIC - ✓ Time travel: read previous versions by version number
```

```
# MAGIC - ✓ Rollback patterns documented (RESTORE, overwrite)
```

```
# MAGIC - ✓ Timestamp-based queries demonstrated
```

What This Proves

- Practical Delta operations (delete, not just append)
- Audit trail and governance (history tracking)
- Time travel for recovery and debugging
- Rollback strategies for production incidents

STEP 5: Performance - Skew vs Broadcast (Spark Tuning)

Objective

Reproduce data skew, show performance impact, fix with AQE/broadcast, and reference Spark UI.

Notebook: notebooks/11_perf_skew_broadcast.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC ## 11_perf_skew_broadcast
# MAGIC Performance tuning demonstration:
# MAGIC – Reproduce data skew
# MAGIC – Measure impact on Spark stages
# MAGIC – Fix with AQE and broadcast
# MAGIC – Compare via Spark UI

# COMMAND -----

from pyspark.sql import functions as F
import time

# COMMAND -----

# MAGIC %md
# MAGIC ### 1. Create Skewed Dataset

# COMMAND -----

# Create synthetic transaction data with HEAVY SKEW
# 40% of transactions belong to customer_id = 1 (hot key)

print("Generating skewed dataset...")

# Normal customers (60% of data, distributed)
normal_data = spark.range(0, 600_000).select(
    (F.col("id") % 1000).alias("customer_id"),
    F.rand().alias("amount"),
    F.current_date().alias("date")
)

# Skewed customer (40% of data, ONE customer)
skewed_data = spark.range(0, 400_000).select(
    F.lit(1).alias("customer_id"), # All go to customer_id = 1
    F.rand().alias("amount"),
    F.current_date().alias("date")
)

df_transactions_skewed = normal_data.union(skewed_data)
```

```

df_transactions_skewed.cache()

print(f"Total transactions: {df_transactions_skewed.count():,}")

# COMMAND -----

# Verify skew
skew_check = df_transactions_skewed.groupBy("customer_id").count().orderBy(F.desc("count"))
display(skew_check.limit(10))

print(f"Customer 1 has {skew_check.filter('customer_id = 1').select('count').collect()[0].count}")
print("✓ Skew confirmed: 40% of data in one partition")

# COMMAND -----

# Create small customer dimension (1000 customers)
df_customers = spark.range(0, 1000).select(
    F.col("id").alias("customer_id"),
    F.concat(F.lit("Customer_"), F.col("id")).alias("customer_name"),
    (F.rand() * 100).cast("int").alias("age")
)

df_customers.cache()
print(f"Customers: {df_customers.count()}")

# COMMAND -----

# MAGIC %md
# MAGIC ### 2. NAIVE JOIN (No Optimization)

# COMMAND -----

# Disable AQE to see raw skew impact
spark.conf.set("spark.sql.adaptive.enabled", "false")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "false")

print("=== NAIVE JOIN (AQE disabled) ===")
start = time.time()

df_naive = df_transactions_skewed.join(df_customers, "customer_id", "left")
naive_count = df_naive.count() # Force execution

naive_time = time.time() - start

```

```

print(f"✓ Naive join completed")
print(f"  Rows: {naive_count:,}")
print(f"  Time: {naive_time:.2f}s")

# COMMAND -----

# Show explain plan
print("\n=== NAIVE JOIN PLAN ===")
df_naive.explain("formatted")

# COMMAND -----

# MAGIC %md
# MAGIC 📸 **SCREENSHOT SPARK UI**: Go to Spark UI → Jobs → Latest job → Stages
# MAGIC – Look for stage with skewed task distribution (one task takes much longer)
# MAGIC – Save as `docs/screenshots/spark_ui_skew.png`

# COMMAND -----

# MAGIC %md
# MAGIC ### 3. OPTIMIZED JOIN (AQE + Broadcast)

# COMMAND -----

# Enable AQE and broadcast
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionFactor", "5")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes", "256MB")
spark.conf.set("spark.sql.adaptive.autoBroadcastJoinThreshold", "10MB")

print("=== OPTIMIZED JOIN (AQE + Broadcast) ===")
start = time.time()

# Broadcast small dimension explicitly
df_optimized = df_transactions_skewed.join(
    F.broadcast(df_customers),
    "customer_id",
    "left"
)

optimized_count = df_optimized.count()

```

```

optimized_time = time.time() - start

print(f"✓ Optimized join completed")
print(f"  Rows: {optimized_count:,}")
print(f"  Time: {optimized_time:.2f}s")

# COMMAND -----

# Show explain plan with broadcast
print("\n=== OPTIMIZED JOIN PLAN (with broadcast) ===")
df_optimized.explain("formatted")

# Look for "BroadcastHashJoin" in the plan

# COMMAND -----

# MAGIC %md
# MAGIC 📸 **SCREENSHOT SPARK UI**: Same view as before
# MAGIC – Should show more balanced task execution
# MAGIC – Save as `docs/screenshots/spark_ui_fixed.png`

# COMMAND -----

# MAGIC %md
# MAGIC ### 4. Performance Comparison

# COMMAND -----

print("\n" + "="*60)
print("PERFORMANCE COMPARISON")
print("="*60)
print(f"Naive join (no AQE, no broadcast):  {naive_time:.2f}s")
print(f"Optimized join (AQE + broadcast):    {optimized_time:.2f}s")
print(f"Speedup: {naive_time/optimized_time:.2f}x faster")
print("="*60)

# COMMAND -----

# MAGIC %md
# MAGIC ### 5. Key Takeaways

# COMMAND -----

```

```

print("""
=== WHAT WE LEARNED ===

1. DATA SKEW SYMPTOMS:
  - One partition has 40% of data (customer_id = 1)
  - One task takes much longer than others in shuffle stage
  - Poor cluster utilization (some executors idle)

2. NAIVE JOIN PROBLEMS:
  - SortMergeJoin with skewed partitions
  - One massive partition causes stragglers
  - Total time dominated by slowest task

3. OPTIMIZATIONS APPLIED:
  - Broadcast join for small dimension (<10MB)
  - Eliminates shuffle for small side
  - AQE skew join handling for large-large joins

4. RESULTS:
  - Broadcast eliminated shuffle entirely
  - All tasks execute in parallel with local data
  - Significant speedup ({speedup:.2f}x in this demo)

5. PRODUCTION RECOMMENDATIONS:
  - Profile joins in Spark UI (Stages → Task Metrics)
  - Broadcast dimensions < 100MB if memory allows
  - Enable AQE for skew handling on large-large joins
  - Consider salting keys for extreme skew cases
""").format(speedup=naive_time/optimized_time if optimized_time > 0 else 0))

# COMMAND -----

# MAGIC %md
# MAGIC ### Summary
# MAGIC - ✓ Reproduced data skew (40% in one key)
# MAGIC - ✓ Measured performance impact (naive join time)
# MAGIC - ✓ Fixed with broadcast join
# MAGIC - ✓ Compared explain plans
# MAGIC - ✓ Documented Spark UI evidence (screenshots)

```

What This Proves

- Ability to diagnose skew in Spark UI

- Understanding of join strategies (shuffle vs broadcast)
- AQE configuration knowledge
- Performance tuning with measurable results

STEP 6: Tests (Data Quality & Reliability)

Objective

Show that data transformations are tested like code, with schema validation, dedup logic, and quality gates.

Test File: tests/test_transforms.py

```
"""
```

```
Unit tests for data transformations.
```

```
Uses pytest + chispa for PySpark DataFrame assertions.
```

```
Run:
```

```
pytest tests/test_transforms.py -v
```

```
"""
```

```
import pytest
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import *
from chispa.dataframe_comparer import assert_df_equality
from datetime import date
```

```
@pytest.fixture(scope="session")
```

```
def spark():
```

```
    """Create Spark session for testing."""
```

```
    return (SparkSession.builder
            .master("local[2]")
            .appName("test_transforms")
            .getOrCreate())
```

```
class TestFXTransform:
```

```
    """Test silver.fx_rates_daily CTE logic."""
```

```
    def test_schema_compliance(self, spark):
```

```
        """Verify output schema matches expected."""
```

```
        # Simulate bronze input
```

```
        bronze_data = [
```

```
            ("2025-01-01", "EUR", "SEK", 11.5, "2025-01-01T10:00:00", "api.test"),
```

```
            ("2025-01-02", "EUR", "USD", 1.08, "2025-01-02T10:00:00", "api.test"),
```

```
        ]
```

```
        df_bronze = spark.createDataFrame(bronze_data,
```

```
            ["as_of_date", "base_currency", "quote_currency", "rate", "ingested_at", "s
```

```
        # Apply transformation
```

```
        df_silver = (df_bronze
```

```
                    .withColumn("pair", F.concat_ws("/", "base_currency", "quote_curre
```

```

        .select("as_of_date", "pair", "rate"))

# Assert schema
expected_schema = StructType([
    StructField("as_of_date", StringType(), True),
    StructField("pair", StringType(), True),
    StructField("rate", DoubleType(), True),
])

assert df_silver.schema == expected_schema, "Schema mismatch"

def test_no_null_primary_keys(self, spark):
    """Ensure PK columns (pair, as_of_date) are never null."""
    bronze_data = [
        ("2025-01-01", "EUR", "SEK", 11.5),
        (None, "EUR", "USD", 1.08), # Bad row
        ("2025-01-03", None, "SEK", 11.6), # Bad row
    ]
    df_bronze = spark.createDataFrame(bronze_data,
        ["as_of_date", "base_currency", "quote_currency", "rate"])

    # Clean (filter nulls)
    df_clean = df_bronze.filter("as_of_date IS NOT NULL AND base_currency IS NOT NU

    # Assert no nulls in PK
    assert df_clean.filter("as_of_date IS NULL OR base_currency IS NULL OR quote_cu

def test_deduplication_logic(self, spark):
    """Verify dedup keeps latest ingestion for duplicate (pair, date)."""
    # Duplicate data: same pair+date, different rates
    bronze_data = [
        ("2025-01-01", "EUR", "SEK", 11.5, "2025-01-01T08:00:00"),
        ("2025-01-01", "EUR", "SEK", 11.6, "2025-01-01T10:00:00"), # Later ingesti
        ("2025-01-02", "EUR", "USD", 1.08, "2025-01-02T10:00:00"),
    ]
    df_bronze = spark.createDataFrame(bronze_data,
        ["as_of_date", "base_currency", "quote_currency", "rate", "ingested_at"])

    # Dedup logic
    df_deduped = (df_bronze
        .withColumn("pair", F.concat_ws("/", "base_currency", "quote_curr
        .withColumn("rn", F.row_number().over(
            F.Window.partitionBy("pair", "as_of_date").orderBy(F.desc("in

```

```

    ))
    .filter("rn = 1")
    .select("as_of_date", "pair", "rate"))

# Expected: only latest rate for 2025-01-01 EUR/SEK
expected_data = [
    ("2025-01-01", "EUR/SEK", 11.6),
    ("2025-01-02", "EUR/USD", 1.08),
]
df_expected = spark.createDataFrame(expected_data, ["as_of_date", "pair", "rate"])

assert_df_equality(df_deduped, df_expected, ignore_row_order=True)

```

```

def test_rate_bounds(self, spark):
    """Ensure rates are positive (data quality check)."""
    bronze_data = [
        ("2025-01-01", "EUR", "SEK", 11.5),
        ("2025-01-02", "EUR", "USD", -1.0), # Invalid
        ("2025-01-03", "EUR", "GBP", 0.0), # Invalid
    ]
    df_bronze = spark.createDataFrame(bronze_data,
        ["as_of_date", "base_currency", "quote_currency", "rate"])

    # Quality check
    df_valid = df_bronze.filter("rate > 0")

    assert df_valid.count() == 1, "Should filter out non-positive rates"

```

```

class TestProductTransform:
    """Test silver.dim_products_api logic."""

    def test_size_class_mapping(self, spark):
        """Verify price > 1000 maps to LARGE, else SMALL."""
        products = [
            (1, "Sofa", 1500.0),
            (2, "Lamp", 50.0),
            (3, "Table", 999.0),
        ]
        df = spark.createDataFrame(products, ["product_id", "title", "price"])

        df_transformed = df.withColumn("size_class",
            F.when(F.col("price") > 1000, "LARGE").otherwise

```

```
# Assert
assert df_transformed.filter("product_id = 1").select("size_class").collect()[0]
assert df_transformed.filter("product_id = 2").select("size_class").collect()[0]
assert df_transformed.filter("product_id = 3").select("size_class").collect()[0]
```

```
# Optional: Great Expectations suite (if you add GE)
# Would go in tests/expectations/fx_rates_suite.json
```

Optional: Great Expectations Suite

If time permits, add `tests/expectations/fx_rates_suite.json` :

```
{
  "expectation_suite_name": "fx_rates_daily",
  "expectations": [
    {
      "expectation_type": "expect_column_values_to_not_be_null",
      "kwargs": {"column": "pair"}
    },
    {
      "expectation_type": "expect_column_values_to_not_be_null",
      "kwargs": {"column": "as_of_date"}
    },
    {
      "expectation_type": "expect_column_values_to_be_between",
      "kwargs": {"column": "rate", "min_value": 0.01, "max_value": 1000}
    },
    {
      "expectation_type": "expect_compound_columns_to_be_unique",
      "kwargs": {"column_list": ["pair", "as_of_date"]}
    }
  ]
}
```

What This Proves

- Data transformations are tested (not just code)
- Schema validation
- Deduplication correctness

- Data quality gates (bounds, nulls)
- Professional testing standards (pytest, chispa)

STEP 7: BI Integration (Downstream Consumption)

Objective

Show understanding of BI consumption patterns and orchestration.

Option A: Databricks SQL Dashboard (No External Creds)

1. Open Databricks SQL workspace
2. Create new query:

```
-- Revenue by Customer Segment (enriched gold data)
```

```
SELECT
```

```
  COALESCE(cust.gender, 'Unknown') AS segment,
```

```
  COUNT(DISTINCT base.receipt_id) AS orders,
```

```
  SUM(base.total_amount) AS revenue,
```

```
  AVG(base.total_amount) AS avg_order_value,
```

```
  SUM(base.total_amount * COALESCE(fx.rate, 1.0)) AS revenue_sek
```

```
FROM gold.baskets_enriched base
```

```
LEFT JOIN silver.dim_customers_api cust ON base.loyalty_id = cust.customer_id
```

```
LEFT JOIN silver.fx_rates_daily fx ON base.date = fx.as_of_date AND fx.pair = 'EUR/SEK'
```

```
GROUP BY segment
```

```
ORDER BY revenue DESC
```

3. Create visualizations:

- Bar chart: Revenue by segment
- Table: Top metrics

4. Screenshot the dashboard → docs/screenshots/bi_dashboard.png

Option B: Power BI Refresh Stub (If You Have Access)

File: ops/powerbi_refresh.py

```
"""
```

Stub for Power BI dataset refresh via REST API.

In production, this would be called after gold layer updates.

Requires:

- Power BI Service workspace
- Dataset ID
- Service principal or user credentials with permissions

```
"""
```

```
import requests
```

```
import os
```

```
def trigger_powerbi_refresh(workspace_id, dataset_id, access_token):
```

```
    """
```

Trigger Power BI dataset refresh.

Docs: <https://learn.microsoft.com/en-us/rest/api/power-bi/datasets/refresh-dataset>

```
    """
```

```
    url = f"https://api.powerbi.com/v1.0/myorg/groups/{workspace_id}/datasets/{dataset_
```

```
    headers = {
```

```
        "Authorization": f"Bearer {access_token}",
```

```
        "Content-Type": "application/json"
```

```
    }
```

```
    response = requests.post(url, headers=headers)
```

```
    if response.status_code == 202:
```

```
        print("✓ Power BI refresh triggered successfully")
```

```
        return True
```

```
    else:
```

```
        print(f"✗ Failed to trigger refresh: {response.status_code}")
```

```
        print(response.text)
```

```
        return False
```

```
# Example usage (would be called from Databricks Job)
```

```
if __name__ == "__main__":
```

```
    # These would come from secrets in production
```

```
    WORKSPACE_ID = os.getenv("POWERBI_WORKSPACE_ID", "your-workspace-id")
```

```
    DATASET_ID = os.getenv("POWERBI_DATASET_ID", "your-dataset-id")
```

```
    ACCESS_TOKEN = os.getenv("POWERBI_ACCESS_TOKEN", "your-token")
```

```

print("""
=== Power BI Refresh Integration ===

In production:
1. Store credentials in Databricks Secrets
2. Call this script from notebook or Databricks Job
3. Trigger after gold layer updates
4. Monitor refresh status via GET endpoint

Example Databricks Job task:
- Type: Python script
- Script: ops/powerbi_refresh.py
- Depends on: gold layer notebook
""")

# Uncomment to actually trigger:
# trigger_powerbi_refresh(WORKSPACE_ID, DATASET_ID, ACCESS_TOKEN)

```

README Section Addition

Add to main [README.md](#):

BI Integration & Orchestration

Databricks SQL Dashboard

- **Query:** ``gold.baskets_enriched`` with customer and FX enrichment
- **Visualizations:** Revenue by segment, order trends
- **Screenshot:** docs/screenshots/bi_dashboard.png

Power BI Refresh (Production Pattern)

In production, we'd automate BI refresh after gold layer updates:

1. **Databricks Workflow:**

Task 1: Ingest API data → bronze

Task 2: Transform → silver

Task 3: Enrich → gold

Task 4: Trigger Power BI refresh (ops/powerbi_refresh.py)

2. ****Power BI Dataset:****

- Connect via Databricks SQL connector
- Import or DirectQuery mode
- Refresh triggered via REST API

3. ****Alternatives:****

- Databricks Lakeview dashboards (no external tool needed)
- Tableau with Databricks connector
- Looker with Databricks integration

What This Proves

- BI consumption patterns
- Downstream orchestration awareness
- API-driven refresh automation
- Production workflow thinking

COMPLETE README UPDATE

Add this section to README.md :

48-Hour Interview Add-on Extension

Overview

This repo includes a ****48-hour extension**** demonstrating advanced data engineering skill

What This Proves

Interview Topic	Artifact	Location
APIs & Data Ingestion	Pagination, retries, schema-on-write	<code>`notebooks/07_ingest_a`</code>
SQL & Databases	CTEs, PK/FK, deduplication	<code>`notebooks/08_silver_api_transform.sql`</code>
Databricks & Architecture	Metadata-driven joins, broadcast hints	<code>`notebooks/09_me`</code>
Delta Lake	Delete, history, time travel	<code>`notebooks/10_delta_time_travel.py`</code>
Performance & Spark	Skew diagnosis, AQE, broadcast	<code>`notebooks/11_perf_skew_broadc`</code>
Testing & Quality	pytest + chispa, Great Expectations	<code>`tests/test_transforms.py`</code>
BI Integration	Dashboard queries, API refresh	SQL dashboard + <code>`ops/powerbi_refres`</code>

Run Order

1. ``07_ingest_api_data.py`` → Fetch DummyJSON + Frankfurter → bronze
2. ``08_silver_api_transform.sql`` → Clean with CTEs → silver
3. ``09_metadata_joins.py`` → Config-driven joins → gold
4. ``10_delta_time_travel.py`` → Governance demo
5. ``11_perf_skew_broadcast.py`` → Performance tuning
6. ``pytest tests/test_transforms.py`` → Run tests locally

Screenshots

- [API Validation (Postman)](docs/screenshots/api_validation.png)
- [Delta History](docs/screenshots/delta_history.png)
- [Spark UI - Skew](docs/screenshots/spark_ui_skew.png)
- [Spark UI - Fixed](docs/screenshots/spark_ui_fixed.png)

What I'd Add with More Time

- Unity Catalog fine-grained permissions (row/column security)
- Full Databricks Workflow with retries, alerts, and email notifications
- CI/CD pipeline with dbt tests and automated deployments
- Monitoring dashboard for data quality metrics

Time Budget Breakdown

Phase	Tasks	Time
Day 1, Morning (4h)	API validation in Postman, export collection, create 07_ingest_api_data.py , test DummyJSON + Frankfurter ingestion	4h
Day 1, Afternoon (4h)	Create 08_silver_api_transform.sql with CTEs, create config/joins.yml , build 09_metadata_joins.py join engine	4h
Day 2, Morning (4h)	Create 10_delta_time_travel.py , capture screenshots, create 11_perf_skew_broadcast.py , capture Spark UI screenshots	4h
Day 2, Afternoon (4h)	Write tests/test_transforms.py , run pytest, update README, polish documentation, final review	4h
Total		16h (2 days)

Final Deliverables Checklist

- ☐ ops/postman_collection.json (API validation exports)
- ☐ notebooks/07_ingest_api_data.py (ingestion)
- ☐ notebooks/08_silver_api_transform.sql (CTE transforms)
- ☐ config/joins.yml (join metadata)
- ☐ notebooks/09_metadata_joins.py (join engine)
- ☐ notebooks/10_delta_time_travel.py (governance)
- ☐ notebooks/11_perf_skew_broadcast.py (performance)
- ☐ tests/test_transforms.py (unit tests)
- ☐ ops/powerbi_refresh.py (BI stub - optional)
- ☐ docs/screenshots/api_validation.png
- ☐ docs/screenshots/delta_history.png
- ☐ docs/screenshots/spark_ui_skew.png
- ☐ docs/screenshots/spark_ui_fixed.png

- ☐ [README.md](#) updated with new section
- ☐ Branch `feature/interview-demo` pushed to GitHub

Talking Points for Follow-up

When you send this to the interviewer or mention it in follow-up:

"I extended the IKEA lakehouse demo to address the technical topics from our interview:

- **API Ingestion:** Validated and ingested data from DummyJSON (retail entities with pagination) and Frankfurter (FX rates) with retry logic and explicit schemas.
- **SQL Modeling:** Built silver transforms using CTEs with explicit primary/composite keys and referential integrity examples.
- **Architecture:** Implemented a metadata-driven join engine (YAML-configured) that scales to 10-20 tables without code changes.
- **Delta Governance:** Demonstrated DELETE, DESCRIBE HISTORY, and time travel with rollback patterns.
- **Performance:** Reproduced data skew (40% in one partition), measured the impact, and fixed it with broadcast joins and AQE. Spark UI screenshots included.
- **Testing:** Added pytest + chispa tests for transformation logic (schema, dedup, quality gates).
- **BI Integration:** Created Databricks SQL dashboard and documented Power BI refresh automation pattern.

Everything is reproducible from the README, with screenshots showing validation-before-coding, Delta history, and performance improvements. If you'd like, I can extend this further with Unity Catalog permissions or a full Databricks Workflow."

Next Steps After Implementation

1. **Create branch:** `git checkout -b feature/interview-demo`
2. **Implement notebooks 07-11** following the code above
3. **Run all notebooks** in Databricks
4. **Capture screenshots** (Postman, Delta history, Spark UI)
5. **Run tests:** `pytest tests/test_transforms.py -v`

6. **Update README** with the new section

7. **Commit and push:**

```
git add .  
git commit -m "Add interview extension: API ingestion, CTEs, metadata joins, Delta  
git push origin feature/interview-demo
```

8. **Send to interviewer** with link to branch and screenshots

Total Implementation Time: 1-2 days

Complexity: Moderate (leverages existing patterns)

Impact: High (proves 7 key DE competencies with concrete artifacts)